

Perbandingan Naive Bayes, Decision Tree, dan K-Nearest Neighbor

untuk Klasifikasi Objek Langit (SDSS17)

Tujuan Penelitian

Notebook ini membandingkan tiga algoritma klasifikasi *fundamental* — **Naive Bayes (GaussianNB)**, **Decision Tree**, dan **K-Nearest Neighbor (KNN)** — pada dataset SDSS17 (Galaxy / Star / QSO), dengan memperhatikan lima celah penelitian (research gap) berikut:

1. **Dominansi algoritma kompleks** → studi ini fokus mengevaluasi algoritma klasik secara berdampingan, bukan ensemble/deep learning.
2. **Ketidakseimbangan data** → ditangani dengan **SMOTE**, dan dibandingkan performa model **sebelum vs sesudah** SMOTE (khususnya recall kelas minoritas: STAR & QSO).
3. **Ketergantungan pada fitur `redshift`** → dilakukan eksperimen ablasi (dengan vs tanpa `redshift`) untuk melihat stabilitas model.
4. **Transparansi vs akurasi** → dibahas interpretabilitas masing-masing model (feature importance Decision Tree, struktur pohon, parameter probabilitas Naive Bayes, permutation importance untuk KNN).
5. **Pencegahan data leakage pada SMOTE** → SMOTE **hanya** diterapkan pada data latih (train), dan dipasang **di dalam pipeline `imblearn`** sehingga pada setiap fold Cross-Validation, SMOTE hanya "melihat" fold latih, tidak pernah menyentuh data validasi/test.

1. Setup & Instalasi

Jalankan cell di bawah ini terlebih dahulu. `imbalanced-learn` (untuk SMOTE)

```
!pip install -q imbalanced-learn kagglehub
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import time

from sklearn.model_selection import train_test_split, StratifiedKFold, GridSearchCV, cross_val_score
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
    classification_report, confusion_matrix, ConfusionMatrixDisplay,
    accuracy_score, precision_recall_fscore_support, f1_score, recall_score
)
from sklearn.inspection import permutation_importance

from imblearn.over_sampling import SMOTE
from imblearn.pipeline import Pipeline as ImbPipeline

sns.set_style("whitegrid")
plt.rcParams["figure.figsize"] = (8, 5)

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)
```

2. Memuat Dataset

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
DATA_PATH = '/content/drive/MyDrive/Dataset/star_classification.csv'

df = pd.read_csv(DATA_PATH)

assert df.shape[0] == 100000, f"🚨 Jumlah baris tidak sesuai! Hanya terdapat {df.shape[0]}."
assert df.isnull().sum().sum() == 0, f"🚨 Ada missing value yang tidak diharapkan!"
```

```
print("✅ Data terverifikasi lengkap:", df.shape)
```

✅ Data terverifikasi lengkap: (100000, 18)

3. Eksplorasi Data Awal (EDA)

Dataset SDSS17 berisi 100.000 observasi dengan 17 kolom fitur + 1 kolom target (`class`), yang berisi tiga label: **GALAXY**, **STAR**, dan **QSO** (quasar).

```
print("Ukuran data:", df.shape)
df.info()
```

```
Ukuran data: (100000, 18)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100000 entries, 0 to 99999
Data columns (total 18 columns):
#   Column                Non-Null Count  Dtype
---  -
0   obj_ID                 100000 non-null  float64
1   alpha                  100000 non-null  float64
2   delta                  100000 non-null  float64
3   u                       100000 non-null  float64
4   g                       100000 non-null  float64
5   r                       100000 non-null  float64
6   i                       100000 non-null  float64
7   z                       100000 non-null  float64
8   run_ID                 100000 non-null  int64
9   rerun_ID               100000 non-null  int64
10  cam_col                100000 non-null  int64
11  field_ID               100000 non-null  int64
12  spec_obj_ID            100000 non-null  float64
13  class                  100000 non-null  object
14  redshift                100000 non-null  float64
15  plate                  100000 non-null  int64
16  MJD                    100000 non-null  int64
17  fiber_ID               100000 non-null  int64
dtypes: float64(10), int64(7), object(1)
memory usage: 13.7+ MB
```

```
df.describe().T
```

	count	mean	std	min	25%	50%	75%	max	
obj_ID	100000.0	1.237665e+18	8.438560e+12	1.237646e+18	1.237659e+18	1.237663e+18	1.237668e+18	1.237681e+18	
alpha	100000.0	1.776291e+02	9.650224e+01	5.527828e-03	1.275182e+02	1.809007e+02	2.338950e+02	3.599998e+02	
delta	100000.0	2.413530e+01	1.964467e+01	-1.878533e+01	5.146771e+00	2.364592e+01	3.990155e+01	8.300052e+01	
u	100000.0	2.198047e+01	3.176929e+01	-9.999000e+03	2.035235e+01	2.217914e+01	2.368744e+01	3.278139e+01	
g	100000.0	2.053139e+01	3.175029e+01	-9.999000e+03	1.896523e+01	2.109983e+01	2.212377e+01	3.160224e+01	
r	100000.0	1.964576e+01	1.854760e+00	9.822070e+00	1.813583e+01	2.012529e+01	2.104478e+01	2.957186e+01	
i	100000.0	1.908485e+01	1.757895e+00	9.469903e+00	1.773228e+01	1.940514e+01	2.039650e+01	3.214147e+01	
z	100000.0	1.866881e+01	3.172815e+01	-9.999000e+03	1.746068e+01	1.900460e+01	1.992112e+01	2.938374e+01	
run_ID	100000.0	4.481366e+03	1.964765e+03	1.090000e+02	3.187000e+03	4.188000e+03	5.326000e+03	8.162000e+03	
rerun_ID	100000.0	3.010000e+02	0.000000e+00	3.010000e+02	3.010000e+02	3.010000e+02	3.010000e+02	3.010000e+02	
cam_col	100000.0	3.511610e+00	1.586912e+00	1.000000e+00	2.000000e+00	4.000000e+00	5.000000e+00	6.000000e+00	
field_ID	100000.0	1.861305e+02	1.490111e+02	1.100000e+01	8.200000e+01	1.460000e+02	2.410000e+02	9.890000e+02	
spec_obj_ID	100000.0	5.783882e+18	3.324016e+18	2.995191e+17	2.844138e+18	5.614883e+18	8.332144e+18	1.412694e+19	
redshift	100000.0	5.766608e-01	7.307073e-01	-9.970667e-03	5.451684e-02	4.241733e-01	7.041543e-01	7.011245e+00	
plate	100000.0	5.137010e+03	2.952303e+03	2.660000e+02	2.526000e+03	4.987000e+03	7.400250e+03	1.254700e+04	
MJD	100000.0	5.558865e+04	1.808484e+03	5.160800e+04	5.423400e+04	5.586850e+04	5.677700e+04	5.893200e+04	
fiber_ID	100000.0	4.493127e+02	2.724984e+02	1.000000e+00	2.210000e+02	4.330000e+02	6.450000e+02	1.000000e+03	

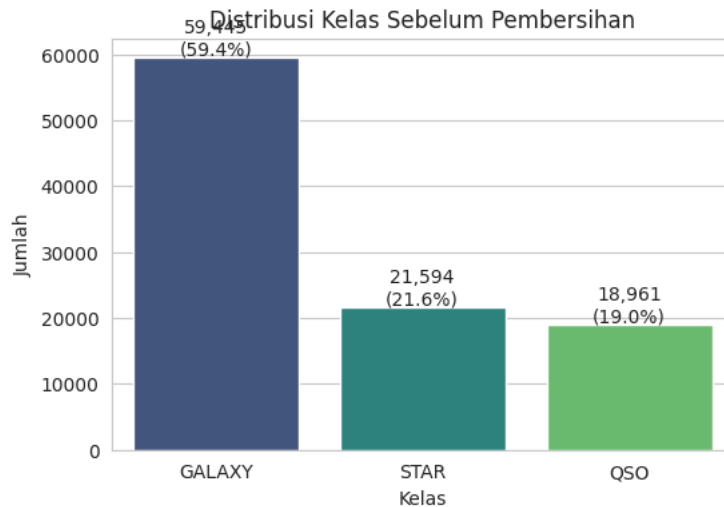
```
print("Jumlah data duplikat:", df.duplicated().sum())
print("\nJumlah missing value per kolom:")
print(df.isnull().sum())
```

```
plt.figure(figsize=(6, 4))
sns.countplot(data=df, x="class", order=df["class"].value_counts().index, palette="viridis")
plt.title("Distribusi Kelas Sebelum Pembersihan")
plt.xlabel("Kelas")
```

```
plt.ylabel("Jumlah")
for i, v in enumerate(df["class"].value_counts().values):
    plt.text(i, v + 500, f"{v:,}\n({v/len(df)*100:.1f}%", ha="center")
plt.show()
```

/tmp/ipykernel_358/42635931.py:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to
 sns.countplot(data=df, x="class", order=df["class"].value_counts().index, palette="viridis")



4. Pembersihan Data

```
sentinel_cols = ["u", "g", "r", "i", "z"]

print("Jumlah baris dengan nilai sentinel (<= -9000) per kolom:")
for c in sentinel_cols:
    n_sentinel = (df[c] <= -9000).sum()
    print(f" {c}: {n_sentinel}")

n_before = len(df)
sentinel_mask = (df[sentinel_cols] <= -9000).any(axis=1)
df_clean = df.loc[~sentinel_mask].copy()
n_after = len(df_clean)

print(f"\nBaris sebelum pembersihan sentinel: {n_before}")
print(f"Baris dibuang (nilai sentinel) : {n_before - n_after}")
print(f"Baris setelah pembersihan sentinel: {n_after}")
```

Jumlah baris dengan nilai sentinel (<= -9000) per kolom:

```
u: 1
g: 1
r: 0
i: 0
z: 1
```

```
Baris sebelum pembersihan sentinel: 100000
Baris dibuang (nilai sentinel) : 1
Baris setelah pembersihan sentinel: 99999
```

```
# Buang duplikat (jika ada) dan kolom non-informatif (ID/metadata instrumen)
df_clean = df_clean.drop_duplicates()

id_cols = ["obj_ID", "spec_obj_ID", "run_ID", "rerun_ID", "cam_col", "field_ID",
           "plate", "MJD", "fiber_ID"]
id_cols_present = [c for c in id_cols if c in df_clean.columns]
df_clean = df_clean.drop(columns=id_cols_present)

print("Kolom tersisa:", list(df_clean.columns))
print("Ukuran data setelah pembersihan:", df_clean.shape)
df_clean.head()
```

Kolom tersisa: ['alpha', 'delta', 'u', 'g', 'r', 'i', 'z', 'class', 'redshift']
 Ukuran data setelah pembersihan: (99999, 9)

	alpha	delta	u	g	r	i	z	class	redshift
0	135.689107	32.494632	23.87882	22.27530	20.39501	19.16573	18.79371	GALAXY	0.634794
1	144.826101	31.274185	24.77759	22.83188	22.58444	21.16812	21.61427	GALAXY	0.779136
2	142.188790	35.582444	25.26307	22.66389	20.60976	19.34857	18.94827	GALAXY	0.644195
3	338.741038	-0.402828	22.13682	23.77656	21.61162	20.50454	19.25010	GALAXY	0.932346
4	345.282593	21.183866	19.43718	17.58028	16.49747	15.97711	15.54461	GALAXY	0.116123

Next steps: [Generate code with df_clean](#) [New interactive sheet](#)

5. EDA Setelah Pembersihan

5.1 Distribusi Kelas (Konfirmasi Data Imbalance)

```
class_counts = df_clean["class"].value_counts()
class_pct = df_clean["class"].value_counts(normalize=True) * 100

summary_imbalance = pd.DataFrame({"Jumlah": class_counts, "Persentase (%)": class_pct.round(2)})
print(summary_imbalance)

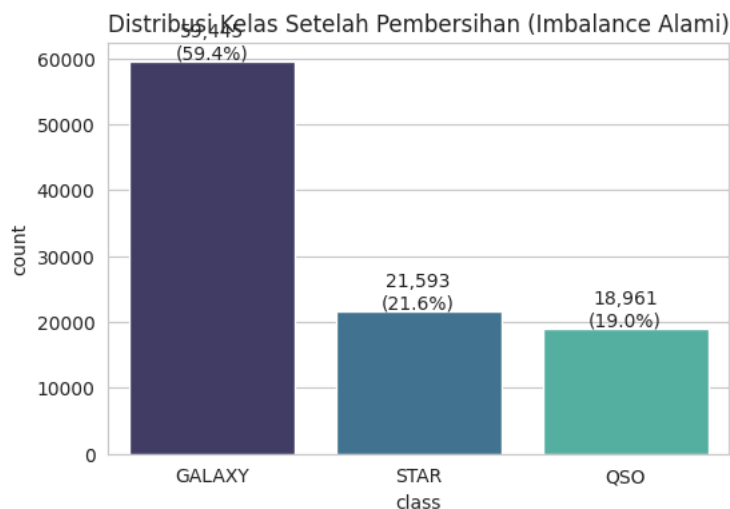
plt.figure(figsize=(6, 4))
sns.countplot(data=df_clean, x="class", order=class_counts.index, palette="mako")
plt.title("Distribusi Kelas Setelah Pembersihan (Imbalance Alami)")
for i, v in enumerate(class_counts.values):
    plt.text(i, v + 500, f"{v:,}\n({v/len(df_clean)*100:.1f}%)", ha="center")
plt.show()
```

	Jumlah	Persentase (%)
class		
GALAXY	59445	59.45
STAR	21593	21.59
QSO	18961	18.96

/tmp/ipykernel_358/1136379697.py:8: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to

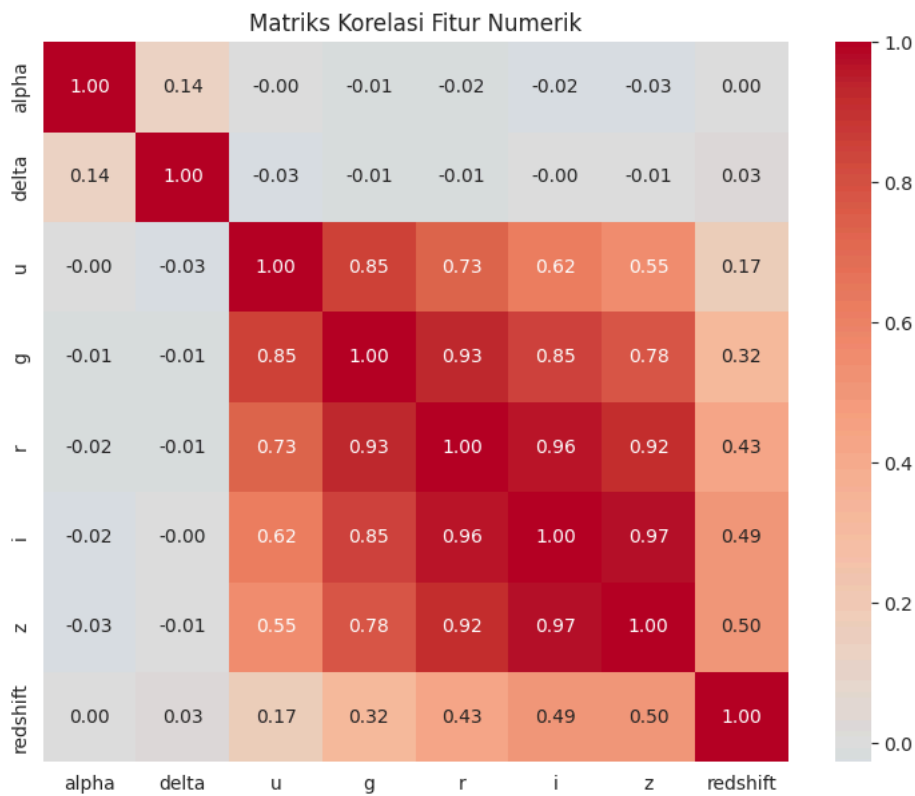
```
sns.countplot(data=df_clean, x="class", order=class_counts.index, palette="mako")
```



5.2 Korelasi Antar Fitur Numerik

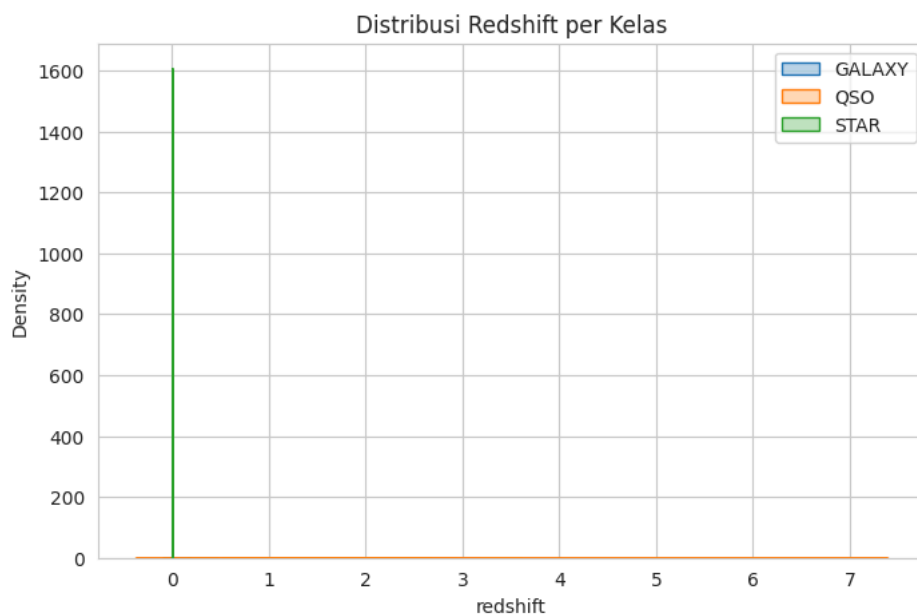
```
numeric_cols = ["alpha", "delta", "u", "g", "r", "i", "z", "redshift"]

plt.figure(figsize=(9, 7))
sns.heatmap(df_clean[numeric_cols].corr(), annot=True, fmt=".2f", cmap="coolwarm", center=0)
plt.title("Matriks Korelasi Fitur Numerik")
plt.show()
```



5.3 Distribusi `redshift` per Kelas

```
plt.figure(figsize=(8, 5))
for cls in df_clean["class"].unique():
    sns.kdeplot(df_clean.loc[df_clean["class"] == cls, "redshift"], label=cls, fill=True, alpha=0.3)
plt.title("Distribusi Redshift per Kelas")
plt.xlabel("redshift")
plt.legend()
plt.show()
```



6. Split Data (Train / Test) — Sebelum SMOTE

Penting (mencegah data leakage, Research Gap 5): data displit menjadi train/test **terlebih dahulu**, sebelum SMOTE dilakukan sama sekali. SMOTE nantinya hanya akan "melihat" `X_train`/`y_train` di dalam pipeline, sehingga data test benar-benar tidak pernah tersentuh proses oversampling.

```
target_col = "class"
feature_cols = [c for c in df_clean.columns if c != target_col]
```

```

X = df_clean[feature_cols].copy()
y_raw = df_clean[target_col].copy()

le = LabelEncoder()
y = le.fit_transform(y_raw)
print("Mapping label:", dict(zip(le.classes_, le.transform(le.classes_))))

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=RANDOM_STATE
)

print("X_train:", X_train.shape, " X_test:", X_test.shape)
print("\nDistribusi kelas di y_train (sebelum SMOTE):")
print(pd.Series(y_train).map(dict(enumerate(le.classes_))).value_counts())

Mapping label: {'GALAXY': np.int64(0), 'QSO': np.int64(1), 'STAR': np.int64(2)}
X_train: (79999, 8) X_test: (20000, 8)

Distribusi kelas di y_train (sebelum SMOTE):
GALAXY    47556
STAR      17274
QSO       15169
Name: count, dtype: int64

```

7. Pipeline Anti-Leakage: Scaling + SMOTE + Model

```

def build_pipeline(model, use_smote=True):
    steps = [("scaler", StandardScaler())]
    if use_smote:
        steps.append(("smote", SMOTE(random_state=RANDOM_STATE)))
    steps.append(("clf", model))
    return ImbPipeline(steps)

```

8. Hyperparameter Tuning dengan Cross-Validation (GridSearchCV)

Setiap model di-tuning menggunakan **Stratified K-Fold Cross-Validation (k=5)** melalui `GridSearchCV`, dengan metrik utama **f1_macro** (menyeimbangkan performa di ketiga kelas, tidak hanya kelas mayoritas GALAXY)

```

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

param_grids = {
    "Naive Bayes": {
        "clf__var_smoothing": np.logspace(-9, -1, 9)
    },
    "Decision Tree": {
        "clf__criterion": ["gini", "entropy"],
        "clf__max_depth": [5, 10, 15, 20, None],
        "clf__min_samples_split": [2, 5, 10],
        "clf__min_samples_leaf": [1, 2, 4],
    },
    "KNN": {
        "clf__n_neighbors": [3, 5, 7, 9, 11, 15],
        "clf__weights": ["uniform", "distance"],
        "clf__metric": ["minkowski", "manhattan"],
    },
}

base_models = {
    "Naive Bayes": GaussianNB(),
    "Decision Tree": DecisionTreeClassifier(random_state=RANDOM_STATE),
    "KNN": KNeighborsClassifier(),
}

```

```

best_estimators = {}
tuning_results = {}

for name, model in base_models.items():
    print(f"=== Tuning {name} (dengan SMOTE di dalam pipeline) ===")
    pipe = build_pipeline(model, use_smote=True)
    grid = GridSearchCV(
        estimator=pipe,
        param_grid=param_grids[name],
        scoring="f1_macro",
        cv=cv,
        n_jobs=-1,
        verbose=0,
    )

```

```

/
t0 = time.time()
grid.fit(X_train, y_train)
elapsed = time.time() - t0

best_estimators[name] = grid.best_estimator_
tuning_results[name] = {
    "best_params": grid.best_params_,
    "best_cv_f1_macro": grid.best_score_,
    "tuning_time_sec": elapsed,
}
print(f"Best params : {grid.best_params_}")
print(f"Best CV f1_macro : {grid.best_score_:.4f}")
print(f"Waktu tuning : {elapsed:.1f} detik\n")

```

```

=== Tuning Naive Bayes (dengan SMOTE di dalam pipeline) ===
Best params : {'clf__var_smoothing': np.float64(1e-06)}
Best CV f1_macro : 0.8930
Waktu tuning : 44.6 detik

```

```

=== Tuning Decision Tree (dengan SMOTE di dalam pipeline) ===
Best params : {'clf__criterion': 'gini', 'clf__max_depth': 10, 'clf__min_samples_leaf': 4, 'clf__min_samples_split': 2}
Best CV f1_macro : 0.9659
Waktu tuning : 1358.4 detik

```

```

=== Tuning KNN (dengan SMOTE di dalam pipeline) ===
Best params : {'clf__metric': 'manhattan', 'clf__n_neighbors': 11, 'clf__weights': 'distance'}
Best CV f1_macro : 0.9303
Waktu tuning : 419.0 detik

```

9. Evaluasi Model pada Test Set

Evaluasi dilakukan pada `X_test`/`y_test` yang **belum pernah** disentuh SMOTE maupun proses tuning, sehingga mencerminkan performa pada distribusi kelas alami/asli.

```

def evaluate_model(name, estimator, X_test, y_test, label_encoder):
    y_pred = estimator.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    prec, rec, f1, _ = precision_recall_fscore_support(y_test, y_pred, average="macro")

    print(f"\n{'='*60}\n{name}\n{'='*60}")
    print(f"Accuracy      : {acc:.4f}")
    print(f"Macro Precision: {prec:.4f}")
    print(f"Macro Recall   : {rec:.4f}")
    print(f"Macro F1       : {f1:.4f}\n")
    print(classification_report(y_test, y_pred, target_names=label_encoder.classes_))

    cm = confusion_matrix(y_test, y_pred)
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=label_encoder.classes_)
    fig, ax = plt.subplots(figsize=(5, 5))
    disp.plot(ax=ax, cmap="Blues", colorbar=False)
    plt.title(f"Confusion Matrix - {name}")
    plt.show()

    return {"accuracy": acc, "precision_macro": prec, "recall_macro": rec, "f1_macro": f1}

test_results = {}
for name, est in best_estimators.items():
    test_results[name] = evaluate_model(name, est, X_test, y_test, le)

```


=====

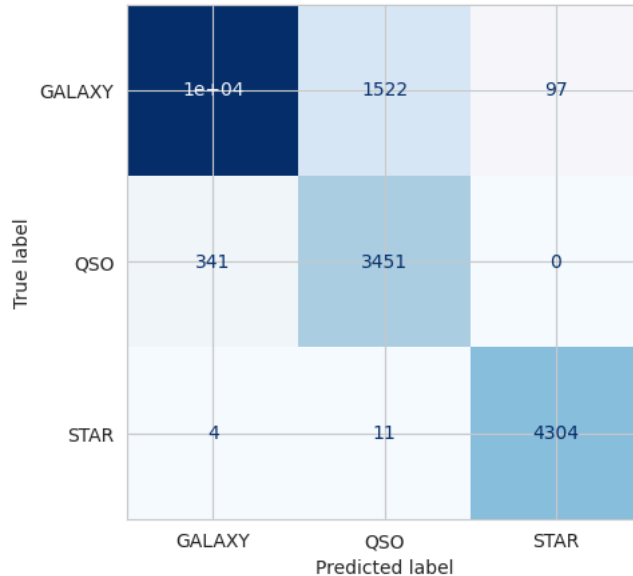
Naive Bayes

=====

Accuracy : 0.9012
 Macro Precision: 0.8793
 Macro Recall : 0.9235
 Macro F1 : 0.8954

	precision	recall	f1-score	support
GALAXY	0.97	0.86	0.91	11889
QSO	0.69	0.91	0.79	3792
STAR	0.98	1.00	0.99	4319
accuracy			0.90	20000
macro avg	0.88	0.92	0.90	20000
weighted avg	0.92	0.90	0.90	20000

Confusion Matrix - Naive Bayes



=====

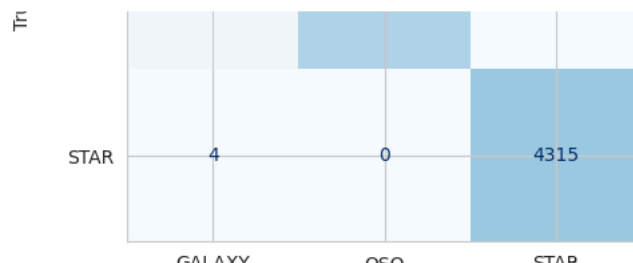
Decision Tree

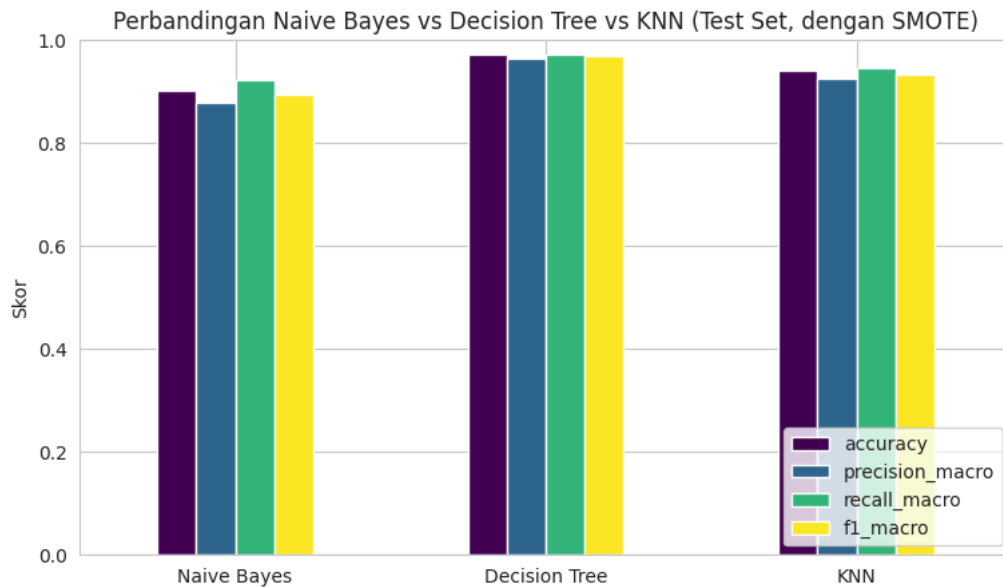
```
results_df = pd.DataFrame(test_results).T
results_df["cv_f1_macro"] = [tuning_results[n]["best_cv_f1_macro"] for n in results_df.index]
results_df["tuning_time_sec"] = [tuning_results[n]["tuning_time_sec"] for n in results_df.index]
results_df = results_df.round(4)
results_df
```

	accuracy	precision_macro	recall_macro	f1_macro	cv_f1_macro	tuning_time_sec
Naive Bayes	0.9012	0.8793	0.9235	0.8954	0.8930	44.6030
Decision Tree	0.9728	0.9655	0.9720	0.9688	0.9659	1358.3982
KNN	0.9398	0.9241	0.9466	0.9342	0.9303	419.0280

Next steps: [Generate code with results](#) [Confusion Matrix](#) [Decision Tree](#)

```
results_df[["accuracy", "precision_macro", "recall_macro", "f1_macro"]].plot(
    kind="bar", figsize=(9, 5), colormap="viridis"
)
plt.title("Perbandingan Naive Bayes vs Decision Tree vs KNN (Test Set, dengan SMOTE)")
plt.ylabel("Skor")
plt.xticks(rotation=0)
plt.ylim(0, 1)
plt.legend(loc="lower right")
plt.show()
```





10. Perbandingan: Dengan SMOTE vs Tanpa SMOTE

```
def recall_per_class(y_true, y_pred, label_encoder):
    recalls = recall_score(y_true, y_pred, average=None)
    return dict(zip(label_encoder.classes_, recalls))

comparison_rows = []

for name, model in base_models.items():
    best_clf_params = {
        k.replace("clf_", ""): v
        for k, v in tuning_results[name]["best_params"].items()
    }

    # Dengan SMOTE (pakai estimator hasil tuning)
    pipe_smote = best_estimators[name]
    y_pred_smote = pipe_smote.predict(X_test)
    rec_smote = recall_per_class(y_test, y_pred_smote, le)

    # Tanpa SMOTE (hyperparameter sama, tanpa langkah SMOTE)
    model_no_smote = base_models[name].__class__(**best_clf_params)
    pipe_no_smote = build_pipeline(model_no_smote, use_smote=False)
    pipe_no_smote.fit(X_train, y_train)
    y_pred_no_smote = pipe_no_smote.predict(X_test)
    rec_no_smote = recall_per_class(y_test, y_pred_no_smote, le)

    for cls in le.classes_:
        comparison_rows.append({
            "Model": name,
            "Kelas": cls,
            "Recall Tanpa SMOTE": round(rec_no_smote[cls], 4),
            "Recall Dengan SMOTE": round(rec_smote[cls], 4),
            "Delta": round(rec_smote[cls] - rec_no_smote[cls], 4),
        })

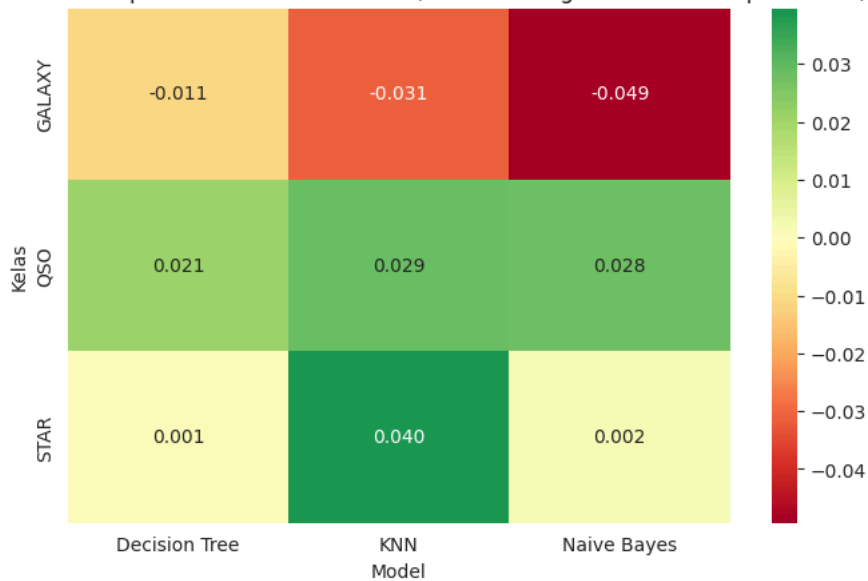
smote_comparison_df = pd.DataFrame(comparison_rows)
smote_comparison_df
```

	Model	Kelas	Recall Tanpa SMOTE	Recall Dengan SMOTE	Delta	
0	Naive Bayes	GALAXY	0.9131	0.8638	-0.0493	
1	Naive Bayes	QSO	0.8821	0.9101	0.0280	
2	Naive Bayes	STAR	0.9942	0.9965	0.0023	
3	Decision Tree	GALAXY	0.9832	0.9717	-0.0114	
4	Decision Tree	QSO	0.9246	0.9459	0.0214	
5	Decision Tree	STAR	0.9986	0.9991	0.0005	
6	KNN	GALAXY	0.9595	0.9281	-0.0314	
7	KNN	QSO	0.9072	0.9357	0.0285	
8	KNN	STAR	0.9363	0.9759	0.0396	

Next steps: [Generate code with smote_comparison_df](#) [New interactive sheet](#)

```
pivot = smote_comparison_df.pivot(index="Kelas", columns="Model", values="Delta")
plt.figure(figsize=(8, 5))
sns.heatmap(pivot, annot=True, fmt=".3f", cmap="RdYlGn", center=0)
plt.title("Perubahan Recall per Kelas Setelah SMOTE (Delta = Dengan SMOTE - Tanpa SMOTE)")
plt.show()
```

Perubahan Recall per Kelas Setelah SMOTE (Delta = Dengan SMOTE - Tanpa SMOTE)



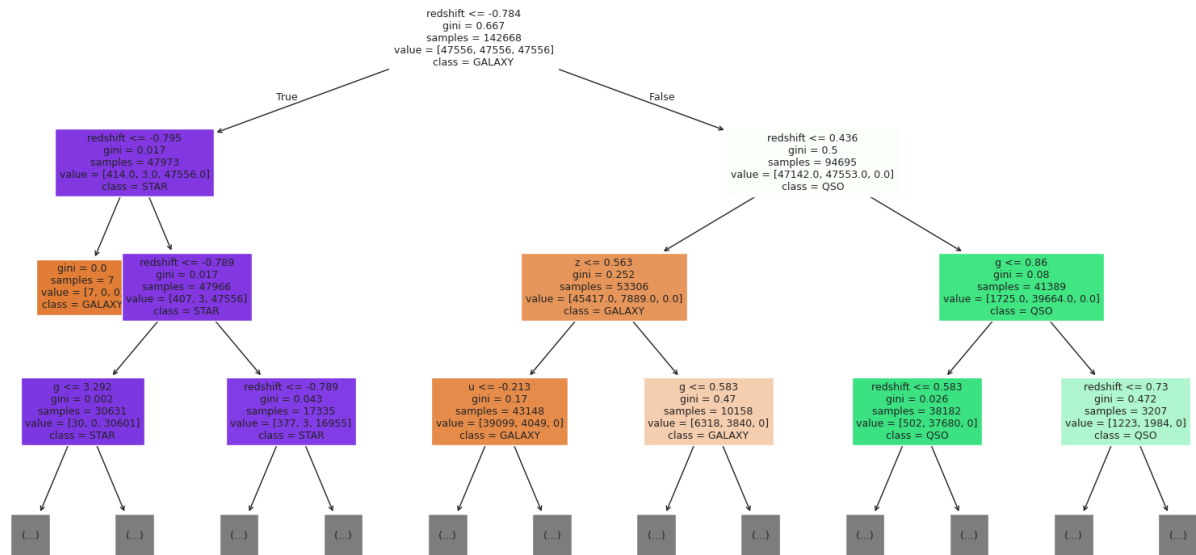
11. Interpretabilitas Model

11.1 Decision Tree

```
dt_model = best_estimators["Decision Tree"].named_steps["clf"]

plt.figure(figsize=(20, 10))
plot_tree(
    dt_model, max_depth=3, feature_names=feature_cols,
    class_names=le.classes_, filled=True, fontsize=9
)
plt.title("Decision Tree (dipotong hingga depth=3 untuk visualisasi)")
plt.show()
```

Decision Tree (dipotong hingga depth=3 untuk visualisasi)



```

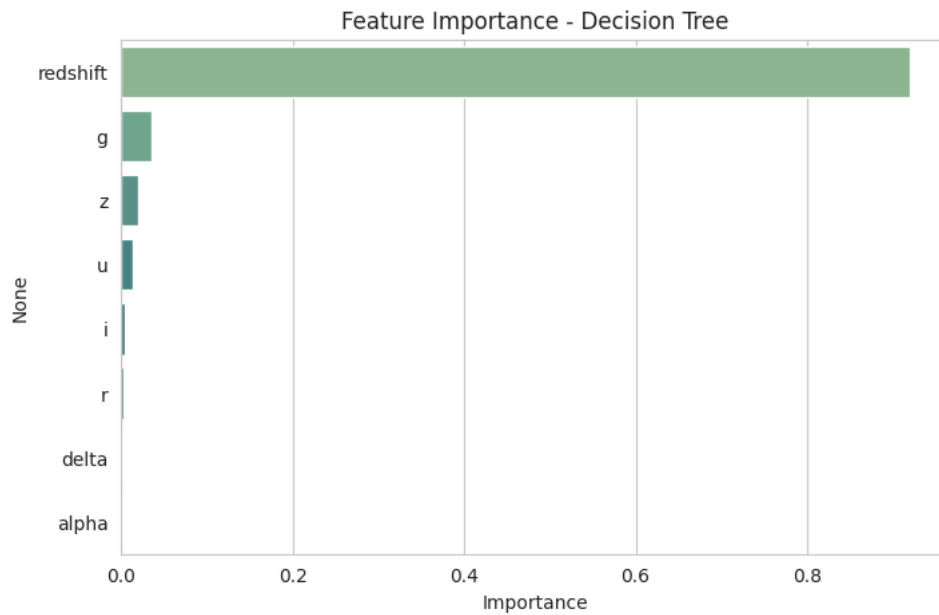
importances = pd.Series(dt_model.feature_importances_, index=feature_cols).sort_values(ascending=False)
plt.figure(figsize=(8, 5))
sns.barplot(x=importances.values, y=importances.index, palette="crest")
plt.title("Feature Importance - Decision Tree")
plt.xlabel("Importance")
plt.show()
print(importances)

```

/tmp/ipykernel_358/4035519654.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to

```
sns.barplot(x=importances.values, y=importances.index, palette="crest")
```



```
redshift    0.919371
g           0.035771
z           0.019807
u           0.014528
i           0.004241
r           0.002792
delta       0.001900
alpha       0.001592
dtype: float64
```

11.2 Naive Bayes

```
nb_model = best_estimators["Naive Bayes"].named_steps["clf"]
```

```
nb_means = pd.DataFrame(nb_model.theta_, columns=feature_cols, index=le.classes_)
print("Rata-rata (mean) fitur per kelas (Naive Bayes, dalam skala standar):")
nb_means
```

Rata-rata (mean) fitur per kelas (Naive Bayes, dalam skala standar):

	alpha	delta	u	g	r	i	z	redshift	
GALAXY	0.002994	-0.032677	0.226249	0.136141	-0.030434	-0.131442	-0.179993	-0.211567	
QSO	0.038461	0.135550	-0.237391	0.144238	0.529257	0.768635	0.848727	1.572867	
STAR	-0.036706	-0.032800	-0.421604	-0.507563	-0.383497	-0.313594	-0.249418	-0.789618	

Next steps: [Generate code with nb_means](#) [New interactive sheet](#)

11.3 KNN

KNN juga tidak punya feature importance bawaan (ia bekerja berbasis jarak antar tetangga). Sebagai gantinya, digunakan **permutation importance**: mengacak satu fitur secara acak dan mengukur seberapa besar penurunan performa model — semakin besar penurunan, semakin penting fitur tersebut.

```

knn_pipe = best_estimators["KNN"]

perm_result = permutation_importance(
    knn_pipe, X_test, y_test, n_repeats=10, random_state=RANDOM_STATE, scoring="f1_macro", n_jobs=-1
)

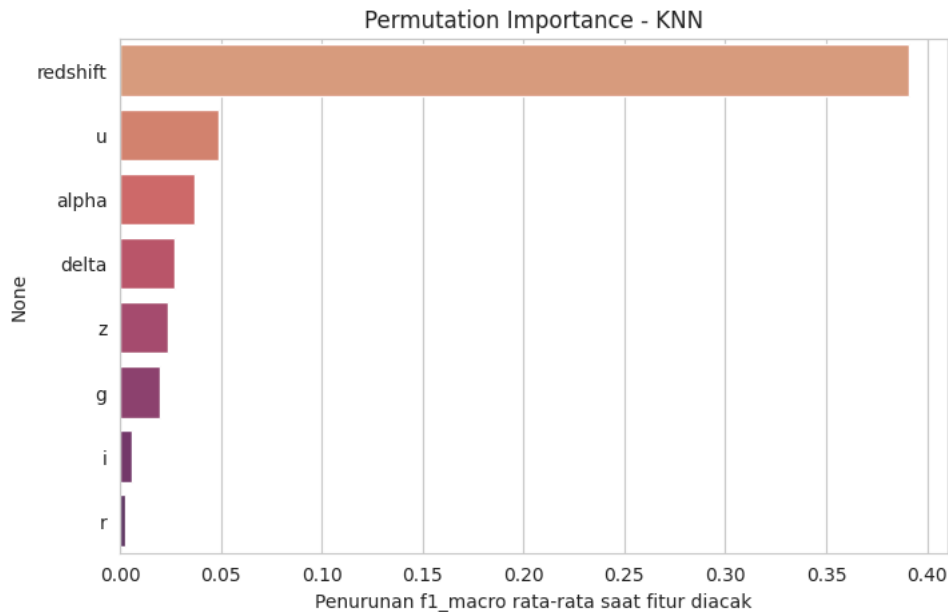
perm_importances = pd.Series(perm_result.importances_mean, index=feature_cols).sort_values(ascending=False)
plt.figure(figsize=(8, 5))
sns.barplot(x=perm_importances.values, y=perm_importances.index, palette="flare")
plt.title("Permutation Importance - KNN")
plt.xlabel("Penurunan f1_macro rata-rata saat fitur diacak")
plt.show()
print(perm_importances)

```

/tmp/ipykernel_358/2066958115.py:9: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to

```
sns.barplot(x=perm_importances.values, y=perm_importances.index, palette="flare")
```



```

redshift    0.390776
u           0.049178
alpha       0.037256
delta       0.027141
z           0.023506
g           0.020065
i           0.006143
r           0.002481
dtype: float64

```

12. Eksperimen Ablasi: Peran Fitur `redshift`

`redshift` membutuhkan pengamatan spektroskopik yang mahal dan tidak selalu tersedia pada survei fotometrik skala besar. Blok ini melatih ulang ketiga model **tanpa** kolom `redshift` (hanya memakai `alpha`, `delta`, `u`, `g`, `r`, `i`, `z`), untuk melihat seberapa jauh model bergantung pada fitur tunggal ini, dan seberapa stabil performanya jika hanya mengandalkan fitur fotometrik.

```

feature_cols_no_redshift = [c for c in feature_cols if c != "redshift"]
X_train_nr = X_train[feature_cols_no_redshift]
X_test_nr = X_test[feature_cols_no_redshift]

ablation_rows = []

for name, model in base_models.items():
    best_clf_params = {
        k.replace("clf_", ""): v
        for k, v in tuning_results[name]["best_params"].items()
    }
    model_nr = base_models[name].__class__(**best_clf_params)
    pipe_nr = build_pipeline(model_nr, use_smote=True)
    pipe_nr.fit(X_train_nr, y_train)
    y_pred_nr = pipe_nr.predict(X_test_nr)

    acc_nr = accuracy_score(y_test, y_pred_nr)
    f1_nr = f1_score(y_test, y_pred_nr, average="macro")

    ablation_rows.append({
        name,
        acc_nr,
        f1_nr
    })

```

```

model : name,
"Accuracy (dengan redshift)": test_results[name]["accuracy"],
"Accuracy (tanpa redshift)": round(acc_nr, 4),
"Delta Accuracy": round(acc_nr - test_results[name]["accuracy"], 4),
"F1 Macro (dengan redshift)": test_results[name]["f1_macro"],
"F1 Macro (tanpa redshift)": round(f1_nr, 4),
"Delta F1 Macro": round(f1_nr - test_results[name]["f1_macro"], 4),
})

ablation_df = pd.DataFrame(ablation_rows)
ablation_df

```

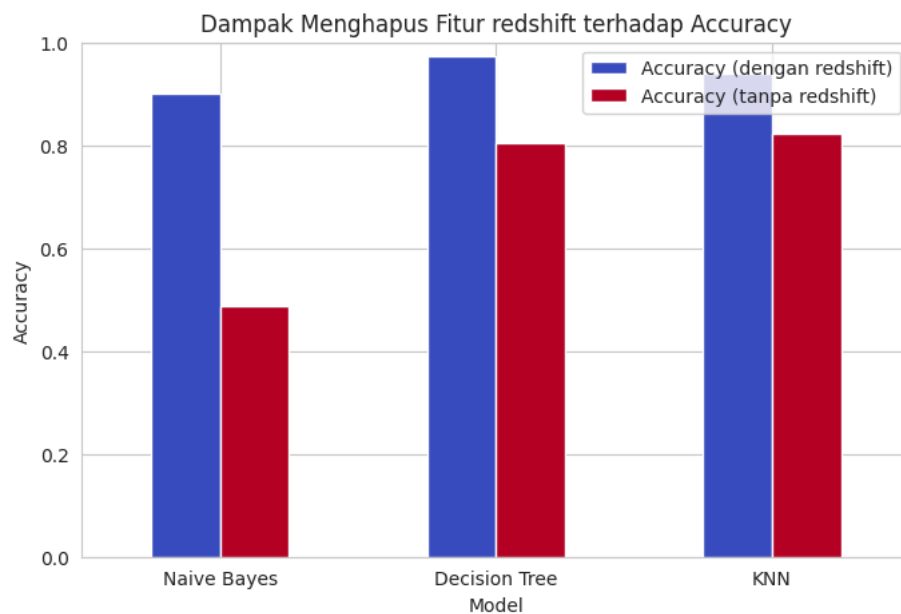
	Model	Accuracy (dengan redshift)	Accuracy (tanpa redshift)	Delta Accuracy	F1 Macro (dengan redshift)	F1 Macro (tanpa redshift)	Delta F1 Macro
0	Naive Bayes	0.90125	0.4878	-0.4135	0.895449	0.4820	-0.4134
1	Decision Tree	0.97275	0.8056	-0.1671	0.968814	0.7685	-0.2003

Next steps: [Generate code with ablation_df](#) [New interactive sheet](#)

```

ablation_plot_df = ablation_df.set_index("Model")
["Accuracy (dengan redshift)", "Accuracy (tanpa redshift)"]
]
ablation_plot_df.plot(kind="bar", figsize=(8, 5), colormap="coolwarm")
plt.title("Dampak Menghapus Fitur redshift terhadap Accuracy")
plt.ylabel("Accuracy")
plt.xticks(rotation=0)
plt.ylim(0, 1)
plt.show()

```



13. Ringkasan & Kesimpulan

```

final_summary = results_df.copy()
final_summary["best_params"] = [str(tuning_results[n]["best_params"]) for n in final_summary.index]
final_summary

```

	accuracy	precision_macro	recall_macro	f1_macro	cv_f1_macro	tuning_time_sec	best_params
Naive Bayes	0.9012	0.8793	0.9235	0.8954	0.8930	44.6030	{'clf__var_smoothing': np.float64(1e-06)}
Decision Tree	0.9728	0.9655	0.9723	0.9688	0.9659	1358.3982	{'clf__criterion': 'gini', 'clf__max_depth': 1...

Next steps: [Generate code with final_summary](#) [New interactive sheet](#)

```

# (Opsional) Simpan hasil ke CSV untuk dilampirkan di laporan
results_df.to_csv("model_comparison_results.csv")
smote_comparison_df.to_csv("smote_vs_no_smote_recall.csv", index=False)
ablation_df.to_csv("redshift_ablation.csv", index=False)
print("Hasil disimpan: model_comparison_results.csv, smote_vs_no_smote_recall.csv, redshift_ablation.csv")

```

[illegible]